09 / 16 — Phase 9 Assessment: Advanced Rendering and Ecosystem

> Framework-specific items are marked. Where the answer key mentions a framework API, accept equivalent correct usage from the framework the learner used. Framework APIs here were checked against docs on 2026-10-22 (React 19.3, Next.js 16.x, React Router 8.x) but move quickly. Judge **reasoning**, not memorized spelling.

Covers: 01 SPA/SSR/SSG/streaming · 02 Hydration internals · 03 Server Components · 04 Client/server boundaries and serialization · 05 Suspense architecture · 06 Concurrent rendering in practice · 07 Modern React APIs · 08 React Compiler · 09 Framework architecture · 10 Routing and nested layouts · 11 Progressive enhancement · 12 Auth boundaries · 13 Forms, actions and mutations · 14 Accessibility and SEO · 15 React API changes across versions

Gate for: Project 9 — SSR/streaming app and Phase 10 — Production engineering.

Time: 4 to 5 hours, closed notes for Parts A–C (about 2 h), open documentation allowed for Parts D–E (about 2–3 h). Complete Parts in order. Do not read the answer key before submitting your own answers.

---

1. Format and scoring overview

Part	Type	Items	Points	Scoring category
A	Conceptual questions	10 × 2.5	25	**Explanation**
B	Code-prediction tasks	5 × 3	15	**Reasoning**
C	Debugging challenges	5 × 5	25	**Debugging**
D	Implementation exercises	3 (10 + 8 + 7)	25	**Implementation**
E	Architecture questions	2 × 5	10	**Reasoning**
			100	

Category totals: **Explanation 25 · Reasoning 25 (B 15 + E 10) · Debugging 25 · Implementation 25.**

Pass criteria (details in §8): **≥ 80 overall** and **≥ 70% in each category** (≥ 17.5 of 25 in each).

---

2. Part A — Conceptual questions (25 points; 2.5 each)

Answer in 3–8 sentences. Awarded: 2.5 = correct, causal, precise; 1.5 = correct but shallow or missing a key distinction; 0.5 = partially correct with a significant error; 0 = wrong or blank.

A1. (01) Define SPA, SSR, SSG and streaming SSR in terms of *when and where HTML is produced*. For each, name one situation where it is the best choice and one where it is wrong.

A2. (02) What is hydration? Explain precisely why a hydration mismatch is a bug rather than a cosmetic issue, name three common causes, and state what React 18/19 does when a mismatch occurs inside a Suspense boundary.

A3. (03) Explain what a Server Component *is* and *is not*. Why are `useState` and event handlers unavailable in them? State two things Server Components remove from the client bundle and one thing they cannot do.

A4. (04) What constraints apply to props passed from a Server Component to a Client Component, and why? Give two examples of values that cannot cross the boundary and a safe alternative for each.

A5. (05) Explain how Suspense boundaries define loading states, what "already revealed content" means in the context of transitions, and how streaming SSR uses boundaries to flush HTML in pieces.

A6. (06) Distinguish *urgent* from *transition* updates. What problem does `useTransition` solve that debouncing does not, and what does it *not* do (e.g., does it make a slow component fast)?

A7. (07, 13) Describe `use`, `useActionState`, `useFormStatus` and `useOptimistic`: the purpose, the signature or return shape of each, and one constraint of each.

A8. (08) What does the React Compiler do, what must your code satisfy for it to be correct, and why is it a *separate tool* from the React version? What does it not replace?

A9. (09, 10) What responsibilities does a React framework add that React does not provide? Explain why the route is the organizing unit and what state a nested layout preserves on navigation, and why.

A10. (11, 12, 14) Choose any **two** of the following and explain them with a concrete failure mode each: (i) why a server function is a public endpoint; (ii) why `<form onSubmit={e => {e.preventDefault(); …}}>` fails pre-hydration; (iii) why streamed content can cause a soft 404; (iv) why `useFormStatus` must be rendered in a child component.

---

3. Part B — Code-prediction tasks (15 points; 3 each)

For each, state the exact observable result **and** explain why. Scoring: 3 = correct result and complete reasoning; 2 = correct result, incomplete reasoning; 1 = partially correct; 0 = wrong.

B1. (Hydration + `useId`). Server renders this component to HTML, the client hydrates it.

```
function Field() {
  const [id] = useState(() => `f-${Math.random().toString(36).slice(2, 7)}`);
  return (<><label htmlFor={id}>Name</label><input id={id} /></>);
}
```

(a) What happens during hydration? (b) What is the user-visible consequence for a screen-reader or click-on-label user? (c) Rewrite it correctly.

B2. (Server/client boundary).

```
// page.tsx (Server Component)
import { Chart } from "./chart";            // "use client"
export default async function Page() {
  const rows = await db.sales.findMany();   // returns Date objects, a BigInt column, and a
class instance Money
  return <Chart rows={rows} onPick={(r) => console.log(r)} />;
}
```

Which props can't be serialized, what error class do you expect, and what exactly would you change?

B3. (Suspense + transitions).

```
function Tabs() {
  const [tab, setTab] = useState("a");
  const [isPending, startTransition] = useTransition();
  return (
    <>
      <button onClick={() => startTransition(() => setTab("b"))}>B</button>
      {isPending && <span>Switching…</span>}
      <Suspense fallback={<p>Loading…</p>}>
        <Panel tab={tab} />   {/* Panel for "b" suspends for 1s on first load */}
      </Suspense>
    </>
  );
}
```

Initially tab "a" is already displayed. The user clicks B. Describe what the screen shows over the next second. Then change `onClick` to `() => setTab("b")` and describe again.

B4. (Forms and actions).

```
const [state, action] = useActionState(async (prev: { n: number }, fd: FormData) => {
  await new Promise((r) => setTimeout(r, 500));
  return { n: prev.n + Number(fd.get("by")) };
}, { n: 0 });

<form action={action}>
  <input name="by" defaultValue="1" />
  <button>Add</button>
  <output>{state.n}</output>
</form>
```

The user presses the button three times in rapid succession (within 100 ms) with `by = 1`, then waits. What is displayed at the end, and at roughly what times does the state update? (State any documented-behavior assumption you rely on.)

B5. (Version behavior).

```
function C() {
  const [a, setA] = useState(0);
  const [b, setB] = useState(0);
  console.log("render", a, b);
  return <button onClick={() => setTimeout(() => { setA(1); setB(1); }, 0)}>go</button>;
}
```

How many renders (excluding the initial one) after one click under (i) React 17 `ReactDOM.render`, (ii) React 18/19 `createRoot`? In the same app under React 19 `<StrictMode>` in development, how many *extra* logs appear for the initial mount, and why?

---

4. Part C — Debugging challenges (25 points; 5 each)

For each: (1) identify the root cause, (2) describe how you would confirm it with a tool or observation, (3) give the fix, (4) say how you would prevent recurrence. Scoring (5): root cause 2, confirmation 1, fix 1, prevention 1.

C1. Soft 404s. After moving a product route to streaming SSR, Search Console reports thousands of "soft 404" URLs for discontinued products. The page:

```
export default function Page({ params }) {
  return (
    <Suspense fallback={<Skeleton />}>
      <Product slug={params.slug} />      {/* awaits db; renders <p>Product not found</p> when
null */}
    </Suspense>
  );
}
```

C2. Cross-user data leak. Users intermittently see another user's name in the header. The header is a Server Component:

```
let cachedUser: User | undefined;                    // module scope
export async function getCurrentUser() {
  if (!cachedUser) cachedUser = await fetchUserFromSession();
  return cachedUser;
}
```

Also describe what other cache layer could produce the same symptom and how you would distinguish the two.

C3. Authorization bypass. A security review shows a non-admin can delete another tenant's project via a replayed request, even though the delete button is hidden for non-admins and the admin layout redirects non-admins.

```
"use server";
export async function deleteProject(id: string) {
  await db.project.delete({ where: { id } });
}
```

C4. Lost focus and silent errors. Keyboard and screen-reader users report that after submitting an invalid form nothing is announced and focus jumps to the top of the page. The code:

```
function Signup({ errors }: { errors: string[] }) {
  return errors.length ? (
    <div className="form-with-errors"><Fields /><ul>{errors.map(e => <li>{e}</li>)}</ul></div>
  ) : (
    <section><Fields /></section>
  );
}
```

(Note also the missing key warning; mention it but prioritize root causes.)

C5. Upgrade regression. After upgrading from React 17 to 19 in a single PR, two issues are reported: (i) "Invalid hook call" in only one app of a monorepo; (ii) a `useEffect` that starts a WebSocket connection appears to connect twice in development and shows duplicate messages. Diagnose both and give a staged plan that would have caught them earlier.

---

5. Part D — Implementation exercises (25 points)

Write code in TypeScript. Compilable, idiomatic React 19; state any framework assumption. Rubrics are in §7.

D1. (10 points) Progressively enhanced, accessible, secure mutation. Implement a "Rename project" feature for a framework with Server Functions (any: Next.js-style `"use server"` or equivalent). Requirements:

1. A Server Function `renameProject(prev, formData)` that: authenticates via `requireSession()`, validates `projectId` (UUID) and `name` (trimmed, 3–80 chars) with a schema, authorizes via a DAL function `renameProjectForUser(session, id, name)` (you only need to write its signature and a short body showing tenant scoping), returns a **discriminated-union** result for validation errors and conflict (name already exists), and **throws** only for unexpected errors; revalidates the affected read.
2. A client form using `useActionState`, a `SubmitButton` using `useFormStatus`, field-level error rendering with `aria-invalid` / `aria-describedby`, an error summary that receives focus on failure, `defaultValue` repopulation after failure, and success announced via `role="status"`.
3. The form must work with JavaScript disabled in the framework's supported configuration (explain in 2–3 sentences what makes this true and what you rely on from the framework).

D2. (8 points) Nested route tree with guards and typed search params. Using React Router framework mode, TanStack Router or Next App Router (your choice; state which), define the route tree for: `/orgs/:orgId/projects` (list, with `?q=&status=&page=`) and `/orgs/:orgId/projects/:projectId`

(detail rendered as a **side panel** while the list remains visible and interactive). Requirements: parse and validate params/search params at the boundary (defaults, clamping, enum whitelist); a layout that loads only cheap shared data; per-segment pending and error UI; a safe `returnTo` redirect for unauthenticated users; focus moved to the panel heading on navigation to the detail route; no authorization **only** in the layout (show the data-layer check).

D3. (7 points) Optimistic list with rollback and idempotency. Implement `<TodoList todos={...} />` with `useOptimistic` and a form action that adds a todo. Requirements: client-generated id used as both the optimistic and the real id (so the DOM node is not remounted); pending styling plus `aria-busy`; the server function takes the client id as an **idempotency key** (show the DB insert with unique constraint handling returning the existing row on replay); on failure the item disappears **and** an error is announced; keyboard-operable; no `useEffect` for the mutation.

---

6. Part E — Architecture questions (10 points; 5 each)

Answer in 250–450 words each. Awarded for clear criteria, explicit trade-offs, a justified decision, and risks/mitigations.

E1. Framework and rendering strategy. A company with a public marketing site (50,000 SEO-critical product pages, hourly price updates by webhook), a logged-in customer portal (dashboards, forms, realtime notifications) and a back-office admin tool is deciding on its front-end platform. Five teams, existing Kubernetes hosting (no Vercel), a security team that requires audited authorization, and an accessibility obligation. Choose a framework strategy (one framework vs several apps; RSC or not), the rendering strategy per area, auth architecture, caching and revalidation approach, and an exit strategy. Identify the top three risks.

E2. Upgrade and governance plan. The company's older apps run React 16.14 (class-heavy, Redux `connect`, enzyme), 17 and 18. A shared component library supports 16 and 17 today. Design a 12-month plan to reach React 19 (and decide whether to adopt RSC), including the library's support policy, sequencing, safety nets, how you will handle silent behavior changes, security patch SLAs, and how you measure progress. State what you deliberately will *not* do.

---

7. Answer key and rubrics

> Do not read this section before submitting your own answers.

7.1 Part A

A1. *Key points:* **SPA**: HTML shell + client-rendered content (HTML produced in the browser after JS runs). Best: authenticated internal dashboards; wrong: public SEO content where first paint/indexing matters. **SSR**: HTML produced on the server per request; best for personalized or frequently changing indexable pages; wrong when server cost/latency dominate and content is static. **SSG**: HTML produced at build time (or regenerated incrementally); best for content that changes rarely; wrong for highly personalized or

fast-changing data (unless combined with client fetch/revalidation). **Streaming SSR**: server flushes a shell early and streams later chunks as data resolves (Suspense boundaries); best when pages have a fast shell and slow secondary sections; caution: status code/headers are committed at first flush, metadata timing, crawler behavior. *Award:* 2.5 for all four with when/where and good/bad cases; deduct for conflating SSR with hydration or SSG with "no JS".

A2. Hydration reuses server-rendered DOM and attaches event handlers/state by running the client render and matching it to the existing DOM. A mismatch means the client's tree differs from the DOM, so React can't safely reuse it: attributes/text can be wrong, handlers can attach to the wrong nodes, accessibility trees can be inconsistent; React must patch or discard and re-render on the client (expensive; flicker). Causes: non-deterministic values (dates, `Math.random`, `useId` replaced by manual ids), browser-only branches (`typeof window`), locale/time-zone differences, invalid HTML nesting that the parser repairs (`<div>` in `<p>`), browser extensions modifying DOM, data differing between server and client. In 18/19, a mismatch inside a Suspense boundary makes React discard the server-rendered content of that boundary and client-render it (recoverable error reported via `onRecoverableError`); an unrecovered mismatch outside a boundary makes the root client-render. Note (lesson 02, executed on 19.3): plain attribute mismatches are only a dev `console.error`; the server value stays in the DOM and `onRecoverableError` is not called. Full marks need the "cannot safely reuse" reasoning and recoverable behavior.

A3. A Server Component renders on the server (at request or build time) to a serializable description (RSC payload) and never ships its code or runs on the client; it can be async and access server resources directly. It is *not* "SSR" (HTML-only) nor a component that re-renders in the browser. `useState`/effects/event handlers require client-side lifecycle and interactivity, which don't exist for components that never mount on the client. Removed from the bundle: the component's own code and its server-only dependencies (e.g., markdown/syntax-highlighting libraries, DB clients). Cannot: use state, effects, browser APIs, handlers; cannot be re-rendered by client state (needs a refetch/navigation); cannot pass non-serializable values to client components.

A4. Props must be serializable by the RSC protocol (primitives, plain objects/arrays, Date, Map/Set, BigInt, Promises, JSX/elements, references to client components, server function references), because they are encoded into the payload sent to the browser. Not serializable: functions (except server functions), class instances with methods/prototypes, unregistered symbols, null-prototype objects, RegExp, DOM nodes. Examples: an `onPick` callback → move the callback into a Client Component or pass a Server Function; a class `Money` instance → pass `{ amountMinor, currency }` and construct in the client; `Date` and `BigInt` do cross the boundary in current React (checked in lesson 04); converting to ISO string / integer minor units is still a good DTO habit. Also mention leakage: everything passed is visible to the user (DTO discipline).

A5. A Suspense boundary declares "if something inside isn't ready, show this fallback instead of the subtree"; the boundary is the unit of loading state. With transitions, React avoids replacing *already revealed* content with a fallback: it keeps the old UI and marks the transition pending (instead of hiding existing content), but a *new* boundary entering the tree for the first time shows its fallback. Streaming SSR: the server renders the shell with fallbacks, flushes it, and later flushes each boundary's HTML plus a tiny script/instruction that swaps the fallback in place; selective hydration hydrates boundaries as they arrive/are interacted with. Credit for noting that one slow data source need not block the whole page.

A6. Urgent updates (typing, clicking, hover) must reflect immediately; transition updates (navigation, filtering results) can be interrupted and deferred. `startTransition`/`useTransition` marks the state

update as non-urgent so React can render it in the background, keep the previous UI interactive, interrupt if a more urgent update arrives, and expose `isPending`. Debouncing delays *when work starts* regardless of CPU capability and still blocks once the work runs; transitions keep work interruptible. It does *not* make slow rendering faster or reduce total work, doesn't help non-React bottlenecks, and an expensive single component render is still atomic. Not suitable for controlled text inputs' own state.

A7. `use(resource)`: reads a Promise (suspends until resolved) or Context in render, may be called conditionally; Promise should be stable (created outside render/cached) and can't be created inside the same render it reads. `useActionState(fn, initial, permalink?)` → `[state, action, isPending]`; `fn(prev, formData)`; sequential invocations; state returned from the action; constraint: first param is previous state. `useFormStatus()` → `{ pending, data, method, action }` from `react-dom`; must be in a child of the `<form>`. `useOptimistic(value, reducer?)` → `[optimistic, add]`; overlay lasts only while the Action is pending; `add` must be called in an Action/transition. (Accept minor naming variation but not wrong tuples.)

A8. The Compiler is a build-time transform (Babel/SWC/bundler plugin) that analyzes components and hooks and inserts fine-grained memoization automatically so values and JSX are reused when inputs haven't changed. Requirements: components/hooks must follow the Rules of React (pure render, no mutation of props/state/hook values, hooks rules); code that violates them may be skipped, or (e.g., mutating props) may compile without any diagnostic and behave incorrectly (lesson 08, executed), which is why the ESLint integration exists. It is separate because it's a compile-time optimization that targets React 17/18/19 via a runtime package (`react-compiler-runtime` plus the `target` option), versioned and released separately. It doesn't replace good architecture (state colocation, avoiding unnecessary work, virtualization), doesn't fix algorithmic problems, doesn't remove the need to profile, and doesn't make effects correct.

A9. A framework adds routing, bundling with server/client graph splitting, a server runtime, data loading orchestration, caching/revalidation, mutation endpoints, adapters/deployment, metadata helpers. The route is the unit because it's where code splitting, data needs, rendering strategy, error/loading boundaries align with the URL. Nested layouts preserve state because the unchanged prefix of the match chain renders as the same element type in the same position; React reconciliation preserves those instances (state, DOM, scroll). `key` or a template remounts. Full marks for tying to reconciliation identity.

A10 (any two). (i) Server functions are HTTP endpoints addressable by id; anyone can craft a request without the UI; so each needs authN/authZ/validation/rate limit/CSRF consideration; failure: IDOR (deleting others' resources). (ii) Without JS, `onSubmit` isn't attached; native submit does GET to the current URL with no handler; data lost or leaked in URL; also unnamed inputs submit nothing. (iii) Status line/headers are sent at first flush; if "not found" is determined inside Suspense after the shell flushes, the response is 200 with "not found" text; the crawler indexes it; fix by resolving existence before flush. (iv) `useFormStatus` reads the nearest ancestor form's status; in the same component that renders the form it has no ancestor form so `pending` is always false; move to a child component. Award 1.25 per item (explanation + concrete failure).

7.2 Part B

B1. (a) `Math.random` generates different ids on server and client; hydration mismatch on the `htmlFor`/`id` attributes: React warns/logs; the client may patch attributes or re-render the boundary (version-sensitive recovery) — **the ids differ until reconciled**. (b) If the DOM keeps the server ids while

React thinks the client id applies (or vice versa), the label/input association can be broken: clicking the label doesn't focus the input; screen readers lose the accessible name; in the repaired case a flash occurs. (c) Use `useId`:

```
function Field() {
  const id = useId();
  return (<><label htmlFor={id}>Name</label><input id={id} /></>);
}
```

(Also acceptable: id passed from the parent deterministically.) *Rubric:* mismatch cause (1), consequence (1), correct fix with `useId` (1).

B2. Unserializable: the `onPick` function (functions can't cross unless server functions); class instance `Money` (prototype lost; error of the "Only plain objects can be passed to Client Components" class; exact text varies by version); `Date` and `BigInt` are supported by the current protocol (lesson 04), so they are not errors, but mapping to a DTO of strings/numbers is still the safer design. Fix: create `ChartRowDTO[]` (`{ id: string; date: string; amountMinor: string; currency: string }`) mapped on the server; move `onPick` into the client component (or accept a server function `pickRow(id)` marked `"use server"` if server action needed). Also note over-fetching and leakage by sending DB rows wholesale. *Rubric:* identifies function and class instance (1.5), appropriate error class/ build/runtime error (0.5), correct DTO + callback placement fix (1).

B3. Transition version: clicking B starts a transition; tab "a" content remains visible; "Switching…" appears (isPending true) for about 1 s; no fallback is shown because "a" content is already revealed and the transition holds back showing a fallback; after Panel b resolves, the UI switches to B and "Switching…" disappears. Non-transition version (`setTab("b")` directly): urgent update; Panel b suspends; the nearest Suspense boundary shows "Loading…" replacing panel a immediately; after 1 s panel b appears. *Rubric:* both correct with the "already revealed" reasoning (3); one correct (1.5).

B4. Model: calls to `action` are **queued and run sequentially**, each receiving the previous result as `prev` (verified by running it: the action starts with `prev.n = 0`, then `1` at ≈500 ms, then `2` at ≈1000 ms). Each action takes 500 ms, so the last one finishes at ≈1500 ms. The key subtlety: the queued actions belong to one Transition, so **the UI does not show the intermediate results**. `isPending` stays true and `<output>` keeps showing **0** until all queued actions have finished, then it jumps straight to **3** (≈1500 ms after the first press). A common wrong answer is "1 at 500 ms, 2 at 1000 ms, 3 at 1500 ms"; the React docs say `isPending` "stays true until all Actions complete and the final state is shown". If invocations ran in parallel from the same `prev`, the result would be 1: also the wrong model. Also: the form fields reset after the action, and `defaultValue="1"` restores "1" in the input. *Rubric:* 3 for sequential model and final 3 with a single visible update after ≈1500 ms; 1.5 for final 3 with the staircase timing or without reasoning.

B5. (i) React 17 legacy: 2 renders ("render 1 0", "render 1 1"). (ii) React 18/19: 1 render ("render 1 1") due to automatic batching. StrictMode dev: React double-invokes the component function during render (extra "render 0 0" logs) on mount and on updates (the second render's logs are dimmed in console in newer DevTools versions), and re-runs effects once (mount/unmount/mount simulation). So initial mount shows two `render 0 0` logs; the click's single batched render also logs twice in dev ("render 1 1" twice). *Rubric:* both counts (1.5), StrictMode double-render explanation (1.5). Accept "double render logs doubled in dev, once in production".

7.3 Part C

C1. *Root cause:* existence is determined inside Suspense after the shell is flushed with a 200 status; later "not found" content cannot change the status → soft 404. *Confirm:* `curl -I https://…/discontinued-slug` returns 200; `curl` the HTML to see the skeleton/“not found” text; Search Console URL Inspection; in dev check the framework's response status. *Fix:* resolve the existence check **before** flush: `const p = await getProductSummary(slug); if (!p) notFound();` outside Suspense, stream secondary data (reviews, recommendations) inside Suspense; return 404/410 for discontinued; remove from sitemaps; request recrawl. *Prevent:* an integration test asserting status codes for missing resources; a lint/review rule "existence + redirects before Suspense"; monitoring 200s with "not found" bodies. *Rubric:* root cause 2, confirm 1, fix 1, prevent 1.

C2. *Root cause:* module-level singleton shared across all requests in the server process; the first request's user is cached and served to subsequent requests (cross-user leak; security incident). *Confirm:* two sessions hitting the same server instance; log session id vs cached user id; grep for module-scope mutable state in server code. *Fix:* remove the module cache; use request-scoped memoization (`cache()` from React in Server Components or per-request context); never memoize identity globally; key any cache by session. Also: **rotate/assess exposure**, add `Cache-Control: private/no-store` . *Other cache layer:* CDN/shared cache or framework data/route cache without session in the key (symptom: not tied to server instance; check `Age` , cache-status headers, `Vary` , whether the page is statically rendered). Distinguish by hitting the origin directly (bypassing CDN), by instance, and by headers. *Prevent:* lint rule/ code review for module-scope state in server modules; automated two-user tests; `server-only` DAL with request-scoped session. *Rubric:* 2 for module scope + cross-request reasoning; 1 for the alternative cache and discrimination.

C3. *Root cause:* authorization missing in the Server Function (a public endpoint); UI gating and layout redirect are not controls; no tenant scoping; `id` unvalidated. *Confirm:* copy the request (cURL) from DevTools and replay with a non-admin cookie and another tenant's id; check server logs. *Fix:*

```
"use server";
export async function deleteProject(formData: FormData) {
  const s = await requireSession();
  const id = z.string().uuid().parse(formData.get("id"));
  await deleteProjectForUser(s, id);      // DAL: role check + tenant-scoped deleteMany +
audit
  revalidatePath("/projects");
}
```

DAL: `if (!s.roles.includes("admin")) throw …; const res = await db.project.deleteMany({ where: { id, tenantId: s.tenantId } }); if (res.count === 0) throw notFound` . *Prevent:* DAL as the only DB access path (lint/dependency rules); an endpoint contract test with principal matrix (anon, other-tenant, viewer, admin); review checklist for each `"use server"` file; DB row-level security as backstop. *Rubric:* 2 root cause (public endpoint, missing object-level authZ and tenant scoping), 1 confirmation by replay, 1 fix, 1 prevention.

C4. *Root cause:* the component renders the form under a different parent element (`div` vs `section`) depending on error state, so React unmounts the old subtree and mounts a new one: the focused element

is destroyed and focus falls to `body`; no error summary receives focus, no live announcement or field association. *Confirm:* in DevTools Console watch `document.activeElement`, React DevTools highlights remount (state reset), use a screen reader or Accessibility panel to see no `aria-invalid` / `aria-describedby`. *Fix:* keep stable structure; render errors alongside (not wrapping differently); error summary with `tabIndex={-1}`, focus it in an effect when errors change, `role="alert"` as secondary; link to fields; `aria-invalid` / `aria-describedby`; add `key` to list items. *Prevent:* RTL tests asserting focus after invalid submit and `getByRole("alert")`; axe + manual SR pass; shared form primitives that implement the pattern (13, 14). *Rubric:* 2 for remount/focus loss + missing announcements; 1/1/1 as above.

C5. (i) *Cause:* two copies of React in the monorepo (a package, e.g., `packages/ui`, depends on `react@18` directly or via hoisting differences) → hooks from a different React instance than the renderer: "Invalid hook call". *Confirm:* `npm ls react react-dom` / `pnpm why react`; log `React` identity from both packages; check bundler resolution. *Fix:* make `react` / `react-dom` peer dependencies in shared packages, align versions, dedupe, add resolutions/overrides carefully. (ii) *Cause:* StrictMode dev simulation (setup→cleanup→setup) exposes an effect without cleanup/unsafe connection logic; production runs once. *Confirm:* check production build; log in setup/cleanup; fix by closing the socket in cleanup and ensuring the effect is idempotent; duplicates in messages arise from two live sockets. *Plan:* stage 17 → 18.2 (legacy root) → `createRoot` → StrictMode fixes → 18.3 warnings → codemods → 19 + types; CI matrix and E2E smoke tests; monorepo dedupe check. *Rubric:* each issue 1.5 (cause+confirm+fix) + staged plan 2.

7.4 Part D (rubrics)

D1. (10)

- Server function (3): authenticates via `requireSession()` (0.5); schema validation with UUID and name constraints, trim (0.75); authorization delegated to DAL with tenant scoping (0.75); discriminated-union result for expected errors, throws for unexpected (0.5); `revalidatePath/Tag` or equivalent (0.5).
- Client form (4): `useActionState(fn, initial)` with `(prev, formData)` signature (0.75); `SubmitButton` in a child using `useFormStatus` (0.75); `aria-invalid` / `aria-describedby` and error text association (0.75); error summary focus on failure (0.75); `defaultValue` repopulation from returned values (0.5); `role="status"` success (0.5).
- PE explanation (2): server-function form renders as a native form that posts to a framework endpoint pre-hydration; name attributes; no `preventDefault` custom handlers; `permalink` (a `useActionState` argument: the URL the browser navigates to if the form is submitted before JS loads) where the page is dynamic; relies on framework support (1 each).
- Code quality (1): types, no `any`, no secrets echoed, no client-supplied identity.

Reference solution sketch:

```
// actions.ts
"use server";
import { z } from "zod";
import { revalidatePath } from "next/cache";
import { requireSession } from "@/lib/auth/session";
import { renameProjectForUser, NameConflictError } from "@/lib/projects/dal";

const schema = z.object({
  projectId: z.string().uuid(),
  name: z.string().trim().min(3, "At least 3 characters").max(80, "At most 80 characters"),
});

export type RenameState =
  | { status: "idle" }
  | { status: "error"; fieldErrors: { name?: string }; values: { name: string } }
  | { status: "conflict"; values: { name: string } }
  | { status: "success" };

export async function renameProject(_p: RenameState, fd: FormData): Promise<RenameState> {
  const s = await requireSession();
  const raw = { projectId: String(fd.get("projectId") ?? ""), name: String(fd.get("name") ??
"") };
  const parsed = schema.safeParse(raw);
  if (!parsed.success) {
    return { status: "error", fieldErrors: { name: parsed.error.flatten().fieldErrors.name?.
[0] }, values: { name: raw.name } };
  }
  try {
    await renameProjectForUser(s, parsed.data.projectId, parsed.data.name);
  } catch (e) {
    if (e instanceof NameConflictError) return { status: "conflict", values: { name: raw.name
} };
    throw e;
  }
  revalidatePath("/projects");
  return { status: "success" };
}

// dal.ts (signature + tenant scoping)
export async function renameProjectForUser(s: Session, id: string, name: string):
Promise<void> {
  if (!(s.roles.includes("editor") || s.roles.includes("admin"))) throw new ForbiddenError();
  try {
    const r = await db.project.updateMany({ where: { id, tenantId: s.tenantId }, data: { name
} });
    if (r.count === 0) throw new NotFoundError();
```

```
  } catch (e) {
    if (isUniqueViolation(e)) throw new NameConflictError();
    throw e;
  }
}
```

```
// rename-form.tsx
"use client";
import { useActionState, useEffect, useRef } from "react";
import { renameProject, type RenameState } from "./actions";
import { SubmitButton } from "./submit-button";

export function RenameForm({ projectId, currentName }: { projectId: string; currentName:
string }) {
  const [state, action] = useActionState(renameProject, { status: "idle" } as RenameState,
`/projects/${projectId}`);
  const summary = useRef<HTMLDivElement>(null);
  const nameError =
    state.status === "error" ? state.fieldErrors.name : state.status === "conflict" ? "That
name is already used." : undefined;
  const defaultName = state.status === "error" || state.status === "conflict" ?
state.values.name : currentName;

  useEffect(() => { if (nameError) summary.current?.focus(); }, [nameError, state]);

  return (
    <form action={action}>
      {nameError && (
        <div ref={summary} tabIndex={-1} role="alert"><h2>There is a problem</h2><a
href="#name">{nameError}</a></div>
      )}
      <input type="hidden" name="projectId" value={projectId} />
      <label htmlFor="name">Project name</label>
      <input id="name" name="name" defaultValue={defaultName} required minLength={3}
maxLength={80}
             aria-invalid={nameError ? true : undefined} aria-describedby={nameError ? "name-
error" : undefined} />
      {nameError && <p id="name-error">{nameError}</p>}
      <SubmitButton>Rename</SubmitButton>
      {state.status === "success" && <p role="status">Project renamed.</p>}
    </form>
  );
}
```

PE explanation: with a framework that renders server-function forms as native forms with identifiers, a pre-hydration submit posts to the server; the function authenticates, validates, mutates, and the response re-

renders or redirects; after hydration React intercepts. We rely on the framework's support, stable `name` attributes, and `permalink` (the `useActionState` argument naming the URL a pre-hydration submit navigates to) for dynamic pages. `useFormStatus` and the focus effect are enhancements only.

D2. (8) Rubric: tree correctness with panel as child route and list persisting (2); validated, typed params/search params with defaults/clamping/whitelist (1.5); layout loads only cheap shared data and independent loaders/fetches start in parallel (1); per-segment pending/error boundaries (1); safe `returnTo` with same-origin validation (1); focus management to panel heading, not on search-param-only changes (0.75); authorization at the data layer (not only layout) (0.75).

Reference sketch (React Router data-router config style; `LoaderArgs` *stands for the loader args type, e.g.* `Route.LoaderArgs` *from generated route types in framework mode):*

```
// routes
[
  { path: "/orgs/:orgId", Component: OrgLayout, loader: orgLoader, ErrorBoundary: OrgError,
    children: [
      { path: "projects", Component: ProjectsLayout /* list + <Outlet/> */, loader:
projectsLoader, ErrorBoundary: ListError,
        children: [
          { index: true, Component: EmptyPanel },
          { path: ":projectId", Component: ProjectPanel, loader: projectLoader, ErrorBoundary:
PanelError },
        ] },
    ] },
];

export async function projectsLoader({ request, params }: LoaderArgs) {
  const s = await requireSessionOrRedirect(request);                // redirect with safe
returnTo
  const q = parseListSearch(new URL(request.url).searchParams);     // page>=1, status
whitelist, q trimmed
  return listProjectsForUser(s, params.orgId!, q);                  // DAL authorizes org
membership
}
export async function projectLoader({ params, request }: LoaderArgs) {
  const s = await requireSessionOrRedirect(request);
  const id = z.string().uuid().safeParse(params.projectId);
  if (!id.success) throw data("Not found", { status: 404 });
  return getProjectForUser(s, params.orgId!, id.data);              // object-level
authorization
}
// ProjectPanel: <h1 tabIndex={-1} ref={headingRef}>; focus in an effect keyed on projectId
```

Layout `OrgLayout` loads only the org name/membership summary. The `safeReturnTo` validates relative paths. Accept equivalent Next/TanStack solutions.

D3. (7) Rubric: `useOptimistic` with reducer producing temp item with client id (1.5); same id sent to server and used by real item (so key stable) (1); server insert idempotent via unique constraint/ `ON CONFLICT DO NOTHING` returning existing (1.5); pending styling + `aria-busy` (0.5); failure: overlay disappears and announced via `role="alert"` /status (1); `addOptimistic` called inside the action before the first `await`, no effect for mutation (0.75); keyboard operable `<form>` / `<button>` with label (0.75).

Reference sketch:

```
"use client";
type Todo = { id: string; title: string; pending?: boolean };

export function TodoList({ todos, addTodo }: { todos: Todo[]; addTodo: (id: string, title:
string) => Promise<{ ok: boolean }> }) {
  const [msg, setMsg] = useState("");
  const [optimistic, add] = useOptimistic(todos, (cur: Todo[], t: Todo) => [...cur, { ...t,
pending: true }]);

  async function action(fd: FormData) {
    const title = String(fd.get("title") ?? "").trim();
    if (!title) return;
    const id = crypto.randomUUID();
    add({ id, title });
    const res = await addTodo(id, title);
    if (!res.ok) setMsg("Could not add the item. It was removed.");   // after await: state
update; wrap in startTransition (see note below)
  }

  return (
    <>
      <ul aria-busy={optimistic.some((t) => t.pending) || undefined}>
        {optimistic.map((t) => <li key={t.id} style={{ opacity: t.pending ? 0.5 : 1 }}>
{t.title}</li>)}
      </ul>
      <form action={action}><label>New todo <input name="title" required /></label>
<button>Add</button></form>
      <p role="alert">{msg}</p>
    </>
  );
}
// server: INSERT ... ON CONFLICT (id) DO NOTHING; then SELECT by id; revalidate before
returning
```

Note: the `msg` set after `await` should be wrapped in `startTransition` per React's documented async-transition limitation (react.dev `useTransition` troubleshooting: "React doesn't treat my state update after await as a Transition"); credit either with the note. `role="alert"` region must be persistent (rendered always).

7.5 Part E (rubrics and model answers)

Score each question out of 5: **criteria and context (1)**, **a justified decision (1.5)**, **trade-offs and alternatives rejected (1)**, **risks and mitigations (1)**, **clarity/feasibility (0.5)**.

E1. Model answer outline.

- *Decomposition:* three areas with different needs: (1) marketing/catalog: SEO, cacheability, scale → SSG/ISR with on-demand revalidation on price webhook (hourly changes mean per-event revalidation, not per-request SSR); (2) customer portal: authenticated, interactive, personalization → SSR/streaming where first-paint matters, otherwise SPA-like; (3) admin tool: no SEO, forms-heavy → SPA/SPA-mode framework or loader/action framework.
- *Frameworks:* default to one framework for (1)+(2) to share design system, auth, and routing conventions; evaluate Next App Router (RSC maturity, ISR) vs React Router framework mode (web-standards forms, simpler cache model). Because hosting is Kubernetes, ISR/data cache needs a shared cache handler across replicas (Redis/S3) or CDN-level caching; if that cost is unacceptable, prefer a framework with simpler caching + CDN purge by webhook. The admin tool may be a separate Vite SPA sharing the design system, to isolate release cadence and risk. Deciding on RSC: adopt for the catalog (bundle size, data colocation), not necessarily for the portal.
- *Auth:* server-side session cookie (`__Host-` , HttpOnly, Secure, SameSite=Lax); DAL as the only DB access path, tenant scoping, DTOs; middleware only optimistic; server functions with the five-step recipe; row-level security backstop; contract tests with principal matrix; security patch SLA for RSC packages.
- *Caching:* explicit layer map; personalized responses `private/no-store` ; tags per entity; webhook → revalidate tags; observability on cache hit ratio.
- *Accessibility:* design system built on headless primitives; axe in CI; manual SR audits; route announcements/focus; native-first forms (PE).
- *Exit strategy:* framework-agnostic domain/DAL packages; thin route adapters; ADRs; E2E tests independent of framework.
- *Top risks:* (1) cache correctness/staleness and cross-user leakage across replicas; (2) framework/RSC upgrade churn and security advisories; (3) team skill gaps in the server/client boundary causing leaks or waterfalls. Mitigations: cache tests, upgrade cadence and canary environments, training + lint rules + review checklists. Accept alternative choices (e.g., React Router framework mode everywhere) if justified against the constraints.

E2. Model answer outline.

- *Baseline:* inventory (React versions, peers, class components count, enzyme usage, test coverage, deprecated API usage by AST scan). Add E2E smoke tests on critical journeys before touching versions.
- *Library first:* support policy "current and previous major" with a transition: step 1 make library work on 16/17/18 (peer `^16.14 || ^17 || ^18`), no new class components, no `defaultProps` on function components, remove string refs/legacy context; step 2 publish a bridge version supporting 17/18/19 (dual CI matrix); step 3 drop 16 with a deprecation notice and migration guide; do **not** use Canary-only APIs.
- *Apps sequencing:* leaf, low-risk apps first to validate; then high-value apps; 16 → 17 (event delegation checks) → 18.2 legacy root → `createRoot` (batching) → StrictMode fixes → 18.3

warnings → codemods → 19 + types. Replace enzyme with RTL progressively (adapter gaps). Parallelize by team using the inventory dashboard.
- *Silent behavior safety nets:* tests for batching-sensitive code (analytics, measurements), StrictMode in CI with warning capture, hydration error counters, Suspense ordering checks, production canary with error-rate and Web Vitals comparison; rollback via feature flag for root switch.
- *RSC decision:* not part of the React upgrade; evaluate per product area (marketing/catalog yes; SPA admin no) after the apps are on 19; requires framework migration with its own plan and security review.
- *Security patching:* SLA (e.g., critical advisories patched within X days) for React server packages and frameworks; Renovate/Dependabot; staging soak.
- *Metrics:* % apps per React version, deprecation warnings count, class component count trend, blocked-library count, error rates, build/test time, Web Vitals.
- *Will not:* mass-rewrite stable class components for style; adopt the Compiler or `useActionState` everywhere as part of the version bump; upgrade everything in one PR; disable StrictMode to silence double effects.

---

8. Scoring rubric and promotion criteria

8.1 Compute your score

Category	Parts	Max	Your score	Percent
Explanation	A	25		
Reasoning	B (15) + E (10)	25		
Debugging	C	25		
Implementation	D	25		
Total		**100**		

8.2 Pass thresholds

- **Overall ≥ 80 / 100**, **and**
- **≥ 70 % in every category** (at least **17.5 / 25** in each of Explanation, Reasoning, Debugging, Implementation).

A high total cannot compensate for a weak category: for example 92 overall with Debugging at 15/25 is **not** a pass.

8.3 Interpreting results

Result	Meaning	Action
Pass (all thresholds met)	Ready for Project 9 and Phase 10	Proceed

Result	Meaning	Action
Pass overall, one category 60–69 %	Near-miss	Review the mapped lessons below; re-attempt the weak category's *different-variant* items (instructor/self-generated) within 7 days; then proceed with Project 9
Overall 70–79 or any category < 60 %	Not yet	Re-study mapped lessons, redo exercises (Levels C–F) in them, retake the full assessment after ≥ 3 days with new variants
Overall < 70	Foundations gap	Return to Phase 2 lessons 25–27 and Phase 9 lessons 01–05 before retrying

8.4 Remediation map

Weak area	Re-study	Practice
Rendering strategy, hydration, streaming	01, 02, 05, Phase 2/25	Break/fix hydration mismatches; measure TTFB vs LCP in a streamed page
Server Components and boundaries	03, 04	Audit one app's props across the boundary; build DTOs
Concurrency, modern APIs, Compiler	06, 07, 08, 13	Re-implement a search UI with transitions; build an optimistic list
Frameworks, routing	09, 10	Build the same nested route tree in two routers
PE, auth, security	11, 12	JS-disabled test suite; authorization matrix tests
Accessibility and SEO	14	Keyboard-only + screen-reader pass; status-code audit
Versions and migration	15	Write an upgrade inventory for a legacy repo

8.5 Promotion criteria (to Project 9 and Phase 10)

A learner is **promoted** when:

1. The scoring thresholds in §8.2 are met.
2. In Part D, each implementation shows **all** of these non-negotiables: server-side validation and authorization (not UI-only), no client-supplied identity, accessible names/error associations, and no mutation in `useEffect`. A missing non-negotiable caps that Part D item at 50 % of its points regardless of other quality.
3. In Part C, the learner correctly names the **trust-boundary** root causes for C2 and C3 (module-scope state; missing object-level authorization). If either is missed, review 12 and redo those two items with variant scenarios before starting Project 9.
4. The learner can explain verbally, in under 3 minutes, one full request lifecycle (match → guard → load → render stream → hydrate → navigate) for the framework used in Project 9, naming where caching and revalidation occur.

Self-assessment honesty: this assessment gates a project that exercises streaming, caching, auth and accessibility together; passing with gaps pushes defects into the project and production-engineering phase.

---

9. Optional extension (not scored)

- Add a no-JS Playwright suite and an axe run to your D1 solution and record results.
- Write an ADR choosing your framework for Project 9, including rejected alternatives and an exit plan (Phase 10/16).
- Produce a one-page "React upgrade readiness" inventory for a legacy repository of your choice using the approach from 15.